Proyecto 2

Aedo Farge, Gian Franco
Colque Ordoñez, Patrick Fabrizio
Sander's Arana, Ralfh Krishna Karl

Arquitectura de Computadoras, noviembre de 2026

Índice

Entrega 1 **2**

1 Organización del trabajo **2**

2 El pipeline de cinco etapas **2**

3 Hazard Unit **3**

4 Programas de prueba **4**

5 Primeros pasos hacia Dual-Issue **8**

6 Conclusiones **10**

A Cómo reproducir las pruebas en Vivado **11**

Entrega 2 **12**

1 Arquitectura Dual-Issue **12**

2 Issue Unit **13**

3 Forwarding, stalls y control de saltos **14**

4 Programas de prueba y waveforms **14**

5 Comparación con el Single-Issue **20**

Entrega 1

1. Organización del trabajo

El proyecto termina comparando tres arquitecturas: (A) el pipeline Single-Issue, (B) el Dual-Issue y (C) el Dual-Issue optimizado. Para que esa comparación sea justa, cada arquitectura se conserva como una versión separada del código y ninguna modifica a la anterior. En esta entrega hay dos: la versión Single-Issue, que es la arquitectura A ya terminada, y la versión Dual-Issue, que parte de ella y contiene el avance hacia B. Todas las waveforms de este informe se obtuvieron simulando los testbenches incluidos en el código, con un reloj de 10 ns; el reset se desactiva a los 22 ns.

2. El pipeline de cinco etapas

Cada instrucción pasa por cinco etapas separadas por registros de pipeline (Tabla 1). Como en cada ciclo hay hasta cinco instrucciones avanzando a la vez, en el caso ideal se termina una instrucción por ciclo (CPI = 1). El procesador soporta `add`, `sub`, `and`, `or`, `xor`, `slt`, sus versiones con inmediato, `lw`, `sw`, `beq`, `jal` y `jalr`.

Tabla 1: Etapas del pipeline.

Etapa	Qué hace	Registro
IF	Lee la instrucción en `PCF` y calcula `PC+4`.	IF/ID
ID	Decodifica, lee `rs1` y `rs2` del Register File y extiende el inmediato.	ID/EX
EX	Opera la ALU, calcula el destino del salto y decide si se toma (`PCSrcE`).	EX/MEM
MEM	`lw` lee y `sw` escribe en la memoria de datos.	MEM/WB
WB	Elige el resultado (ALU, dato leído o `PC+4`) y lo escribe en el Register File.	—

2.1 Datapath

La Figura 1 muestra el datapath. Las señales llevan como sufijo la etapa donde están (`F`, `D`, `E`, `M`, `W`). Además de datos y control, los registros de pipeline propagan la instrucción completa (`InstrD` a `InstrW`); no la necesita el hardware, pero permite ver en las waveforms en qué etapa está cada instrucción.

El multiplexor del PC tiene tres entradas: `00` para `PC+4`, `01` para el destino de `beq` y `jal` (`PCTargetE`) y `10` para `jalr`, cuyo destino es `ALUResultE` con el bit 0 en cero. El Register File escribe en el flanco de bajada del reloj: así, una instrucción en WB escribe en la primera mitad del ciclo y la que está en ID lee el valor nuevo en la segunda mitad, lo que nos ahorra un caso de forwarding. Cuando el registro IF/ID se anula, se carga un NOP real (`addi x0, x0, 0`) en lugar de ceros, para que el controller nunca decodifique una instrucción inválida.

```
// PC: 00 = PC+4, 01 = PCTargetE, 10 = jalr (bit 0 en cero)
mux3 #(WIDTH) pcmux(.d0(PCPlus4F), .d1(PCTargetE), .d2({ALUResultE[31:1], 1'b0}),
                    .s(PCSrcE), .y(PCNextF));

// Register File: escribe en el flanco de bajada
always @(negedge clk)
  if (we3) rf[a3] <= wd3;
```

2.2 Unidad de control

El controller decodifica en ID con `maindec` y `aludec` y hace viajar cada señal con su instrucción hasta la etapa donde se usa. `ImmSrc` se consume en ID; `ALUSrc`, `ALUControl`, `Branch`, `Jump` y `Jalr` en EX;

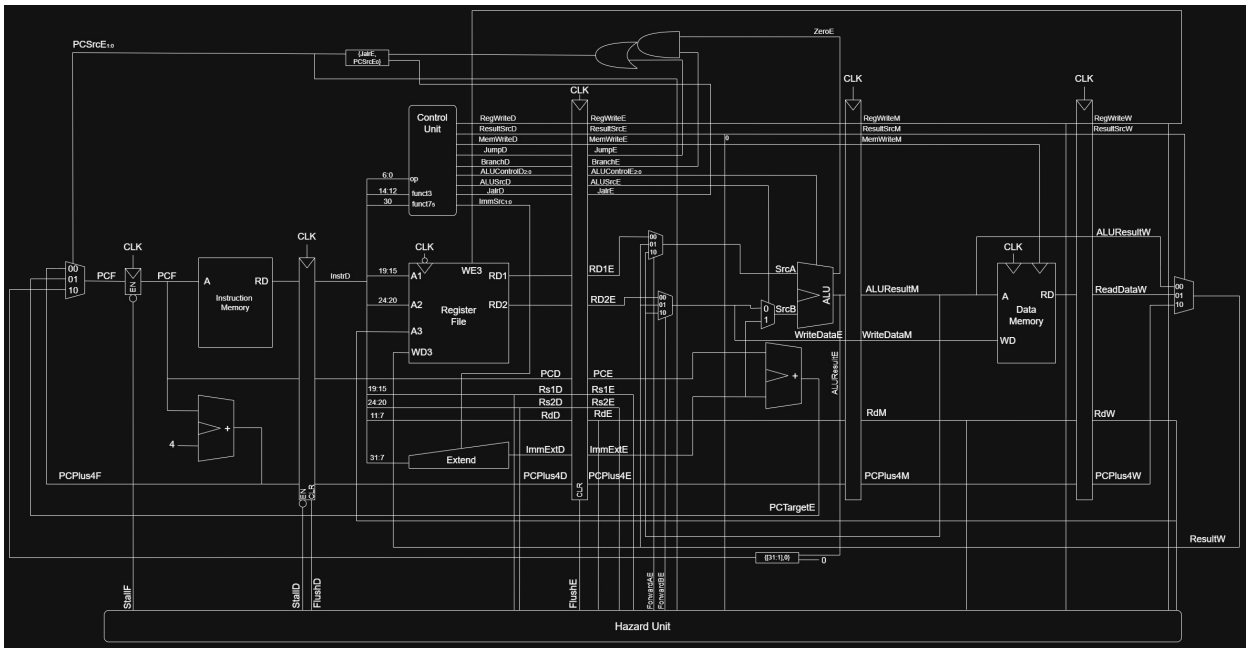


Figura 1: Datapath del pipeline Single-Issue con la Hazard Unit: registros de pipeline, multiplexores de forwarding en EX y señales de stall y flush.

`MemWrite` en MEM, y `RegWrite` y `ResultSrc` en WB. El salto se resuelve en EX con `PCSrcE = {JalrE, (BranchE & ZeroE) | JumpE}`. El registro de control ID/EX se limpia con `FlushE`, de modo que una instrucción anulada no escribe registros ni memoria. Respecto al procesador single-cycle original, añadimos la señal `Jalr` a `maindec` (la palabra de control pasó de 11 a 12 bits) para soportar `jalr`:

```
7'b1100111: controls = 12'b1_00_1_0_10_0_00_0_1; // jalr, en maindec
assign PCSrcE = {JalrE, (BranchE & ZeroE) | JumpE};
```

3. Hazard Unit

La Hazard Unit (`hazardunit.v`) se encarga de los tres tipos de riesgo del pipeline.

Forwarding. Si un registro fuente de la instrucción en EX es el destino de una instrucción en MEM o WB que escribe en el Register File, se le pasa el valor directamente. MEM tiene prioridad porque contiene el resultado más reciente:

```
if      ((Rs1E != 0) && (Rs1E == RdM) && RegWriteM) ForwardAE = 2'b10; // desde MEM
else if ((Rs1E != 0) && (Rs1E == RdW) && RegWriteW) ForwardAE = 2'b01; // desde WB
else                                                ForwardAE = 2'b00;
```

Stall (load-use). Un `lw` recién tiene su dato al final de MEM. Si la instrucción que le sigue lo necesita, se congelan el PC y el registro IF/ID durante un ciclo y se ingresa una burbuja en EX:

```
assign lwStall = ResultSrcE0 & (RdE != 5'b0) & ((Rs1D == RdE) | (Rs2D == RdE));
assign StallF  = lwStall;
assign StallD  = lwStall;
```

Flush. Para instrucciones del formato B o de salto, el salto se decide en EX, pero para entonces ya entraron dos instrucciones (en IF e ID). Si el salto se toma, ambas se anulan, realizando el Flush con los registros IF/ID e ID/EX:

```
assign FlushD = |PCSrcE;
assign FlushE = lwStall | (|PCSrcE);
```

Error corregido: `jalr` no hacía flush

La versión original tenía `assign FlushD = PCSrcE;`. `PCSrcE` es de 2 bits y `FlushD` de 1, así que Verilog se quedaba solo con el bit menos significativo. Para `jalr` (`PCSrcE = 10`) eso daba `FlushD = 0`, y las dos instrucciones siguientes se ejecutaban aunque no debían. Lo corregimos con la reducción OR (`|PCSrcE`), que vale 1 ante cualquier salto tomado. El Programa 4 lo comprueba: con la versión anterior, `x14` y `x15` terminaban en 99; con la corregida quedan en 0. También agregamos `RdE != 0` a `lwStall`, porque un `lw` hacia `x0` no debería frenar el pipeline.

4. Programas de prueba

Cada programa tiene su código Assembly, su archivo `.mem` (verificado contra el ensamblador RISC-V) y un testbench que revisa por sí solo los registros, la memoria y la cantidad de stalls y flushes. La Tabla 2 resume lo obtenido; en las secciones siguientes explicamos cada waveform.

Tabla 2: Resultados de los programas de prueba sobre el pipeline Single-Issue.

Programa	Qué prueba	Qué se observa	Testbench
1 — ISA	Todas las instrucciones, sin dependencias	Sin forwarding, stall ni flush	PASS
2 — Forwarding	Cadena de dependencias RAW	4 ciclos con forwarding	PASS
3 — Stalling	Dependencia load-use	1 stall	PASS
4 — Flushing	`beq`, `jal`, `jalr`	3 flushes	PASS

4.1 Programa 1: instrucciones sin dependencias

Este programa recorre todas las instrucciones que soporta el procesador y las ordena para que ninguna necesite a la Hazard Unit: cada instrucción que produce un valor está al menos a tres instrucciones de la que lo usa, distancia a la que el Register File ya tiene el dato gracias a la escritura en flanco de bajada. `jal`, `jalr` y un `beq` tomado no pueden probarse aquí, porque siempre provocan un flush; esos se verifican en el Programa 4. El testbench comprueba los registros `x1` a `x18`, las dos palabras escritas en memoria y que durante todo el programa no haya ningún forwarding, stall ni flush.

```
addi x4, x0, 3       # x4 = 3
addi x1, x0, 12      # x1 = 12
addi x2, x0, 5       # x2 = 5
addi x3, x0, 8       # x3 = 8
                     # 4 NOP
add  x5, x1, x2      # x5 = 17
sub  x6, x3, x4      # x6 = 5
and  x7, x1, x3      # x7 = 8
or   x8, x2, x4      # x8 = 7
xor  x9, x1, x2      # x9 = 9
slt  x10, x4, x2     # x10 = 1   (3 < 5)
addi x11, x0, -5     # x11 = -5
andi x12, x1, 6      # x12 = 4   (12 & 6)
ori  x13, x2, 8      # x13 = 13  (5 | 8)
xori x14, x3, 15     # x14 = 7   (8 ^ 15)
slti x15, x2, 7      # x15 = 1   (5 < 7)
sw   x1, 0(x0)       # Mem[0] = 12
sw   x2, 4(x0)       # Mem[1] = 5
lw   x16, 0(x0)      # x16 = 12
lw   x17, 4(x0)      # x17 = 5
                     # 3 NOP
beq  x16, x17, salto  # 12 != 5: no se toma
addi x18, x0, 1      # x18 = 1
salto:
```



Figura 2: Programa 1. Arriba, de 20 a 200 ns; abajo, de 195 a 345 ns.

En la Figura 2 se ve el caso ideal del pipeline: cada instrucción baja en diagonal de `InstrF` a `InstrW`, una etapa por ciclo, y el PC avanza de 4 en 4 sin detenerse. `StallF`, `FlushD`, `ForwardAE`, `ForwardBE` y `PCSrcE` se mantienen en 0 durante todo el programa, así que la Hazard Unit nunca interviene. `ResultW` muestra los valores en el orden en que se escriben: 3, 12, 5 y 8 de la inicialización (60–90 ns); 17, 5, 8, 7, 9 y 1 de las instrucciones tipo R, incluido el `slt` (140–190 ns), y −5, 4, 13, 7 y 1 de las instrucciones con inmediato (200–240 ns).

La parte de memoria está en la captura inferior. `MemWrite` sube durante dos ciclos, entre los 235 y 255 ns: son los dos `sw`, que escriben 12 en `Mem[0]` y 5 en `Mem[1]`. A los 270 y 280 ns, `ResultW` muestra 12 y 5, los valores que los `lw` leyeron de esas mismas direcciones. El `beq` llega a EX a los 300 ns; como 12 ≠ 5,

PCSrcE sigue en 00, no hay flush, y addi x18 escribe su 1 a los 330 ns. Los valores 0 y 4 de ResultW hacia los 250 ns son las direcciones que calcularon los sw, y el 7 de los 320 ns es la resta 12 − 5 del beq; ninguno se escribe, porque esas instrucciones tienen RegWrite = 0.

4.2 Programa 2: forwarding

```
addi x1, x0, 5      # x1 = 5
addi x2, x0, 10     # x2 = 10
add  x3, x1, x2     # x3 = 15   (x1 desde WB, x2 desde MEM)
sub  x4, x3, x1     # x4 = 10   (x3 desde MEM)
add  x5, x4, x3     # x5 = 25   (x4 desde MEM, x3 desde WB)
add  x6, x5, x4     # x6 = 35   (x5 desde MEM, x4 desde WB)
```

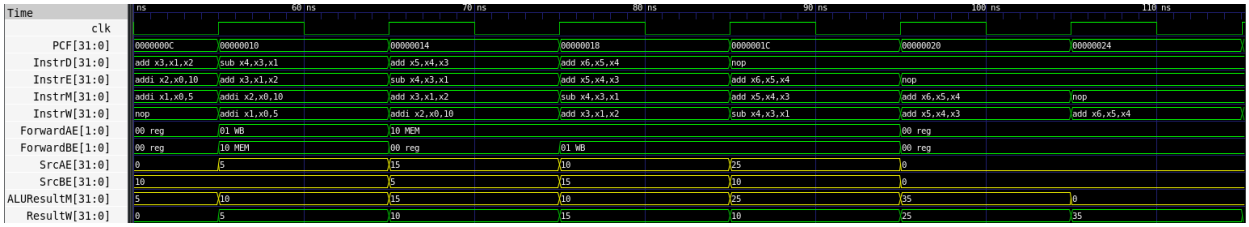


Figura 3: Programa 2, de 50 a 115 ns. En amarillo, las señales agregadas para ver los datos, en decimal. Las instrucciones aparecen en ensamblador y ForwardAE y ForwardBE indican el origen del dato.

Aquí cada instrucción necesita el resultado de la anterior antes de que llegue al Register File. La Tabla 3 resume lo que hace el forwarding en cada ciclo de la Figura 3. Ninguna instrucción se detiene: sin forwarding, cada una habría esperado dos ciclos. ResultW confirma los valores 15, 10, 25 y 35 entre los 80 y 110 ns.

Para ver el dato que realmente usa la ALU agregamos a la waveform SrcAE, SrcBE y ALUResultM. SrcAE y SrcBE son las salidas de los multiplexores de forwarding, que eligen el operando según ForwardAE y ForwardBE:

```
// 00 = registro, 01 = ResultW (WB), 10 = ALUResultM (MEM)
mux3 #(WIDTH) srcamux(.d0(RD1E), .d1(ResultW), .d2(ALUResultM), .s(ForwardAE), .y(SrcAE));
```

La selección la hace el código de la Sección 3, y SrcBE se obtiene igual con ForwardBE. Así, cada operando se comprueba en la Figura 3: a los 60 ns, ForwardAE = 01 hace que SrcAE tome el 5 de ResultW, y ForwardBE = 10 hace que SrcBE tome el 10 de ALUResultM. La suma da el 15 que aparece en ALUResultM a los 70 ns.

Tabla 3: Forwarding observado en el Programa 2.

Tiempo	En EX	ForwardAE	ForwardBE	SrcAE	SrcBE	Origen de los operandos
60 ns	add x3, x1, x2	01	10	5	10	x1 desde WB, x2 desde MEM
70 ns	sub x4, x3, x1	10	00	15	5	x3 desde MEM, x1 del registro
80 ns	add x5, x4, x3	10	01	10	15	x4 desde MEM, x3 desde WB
90 ns	add x6, x5, x4	10	01	25	10	x5 desde MEM, x4 desde WB

4.3 Programa 3: stalling por load-use

```
addi x1, x0, 0      # x1 = 0
addi x2, x0, 10     # x2 = 10
sw   x2, 0(x1)      # Mem[0] = 10
lw   x3, 0(x1)      # x3 = 10
add  x4, x3, x2     # x4 = 20   <- usa x3 justo despues del lw: stall
sub  x5, x4, x2     # x5 = 10
add  x6, x5, x3     # x6 = 20
```

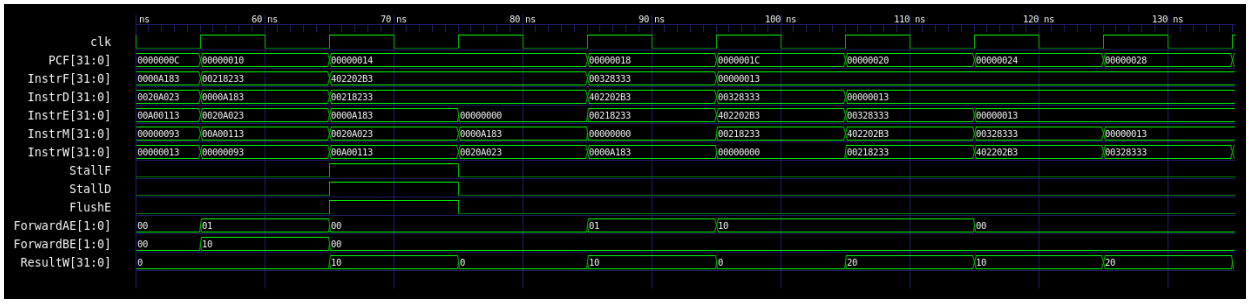


Figura 4: Programa 3, de 50 a 135 ns.

La Figura 4 muestra el único stall del programa. A los 60 ns, el `sw` recibe por forwarding desde MEM el dato que va a guardar (`ForwardBE = 10`). A los 70 ns el `lw x3` está en EX y el `add x4, x3, x2` en ID; como `x3` recién estará listo al final de MEM, la Hazard Unit levanta a la vez `StallF`, `StallD` y `FlushE`. El efecto se ve claro a los 80 ns: `PCF` sigue en `0x14` e `InstrD` sigue siendo el `add`, porque ambos quedaron congelados, mientras `InstrE` vale 0, que es la burbuja insertada.

Un ciclo después, a los 90 ns, el `add` entra a EX y toma `x3 = 10` desde WB (`ForwardAE = 01`), porque el `lw` ya leyó la memoria. Las dos instrucciones siguientes obtienen su operando desde MEM (`ForwardAE = 10`) sin más esperas. Un ciclo de stall es el costo mínimo de una dependencia load-use en este pipeline.

4.4 Programa 4: flushing

```
      addi x1, x0, 5
      addi x2, x0, 5
      addi x3, x0, 7
      beq  x1, x3, t1     # NO se toma (5 != 7)
      addi x4, x0, 1      # x4 = 1
      beq  x1, x2, t1     # se toma -> flush
      addi x5, x0, 99     # anulada
      addi x6, x0, 99     # anulada
t1:   addi x7, x0, 2
      jal  x8, t2         # x8 = 40 -> flush
      addi x9, x0, 99     # anulada
      addi x10, x0, 99    # anulada
t2:   addi x11, x0, 3
      addi x12, x0, 68
      jalr x13, 0(x12)    # x13 = 60 -> flush
      addi x14, x0, 99    # anulada
      addi x15, x0, 99    # anulada
t3:   addi x16, x0, 4
      sw   x16, 0(x0)
fin:  beq  x0, x0, fin
```

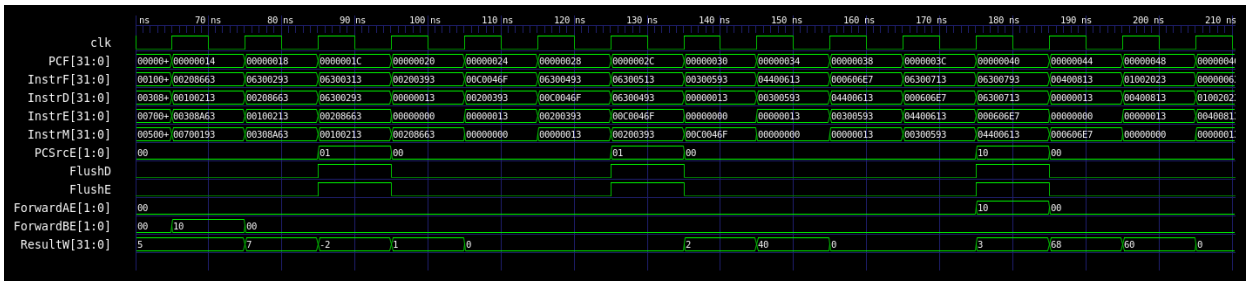


Figura 5: Programa 4, de 60 a 210 ns.

El programa recorre los tres tipos de salto. En la Figura 5, a los 70 ns el primer `beq` está en EX y recibe `x3` por forwarding (`ForwardBE = 10`); como $5 \neq 7$, `PCSrcE` queda en `00` y no pasa nada: `addi x4` sigue su camino.

A los 90 ns el segundo `beq` sí se toma. `PCSrcE` pasa a `01` y se activan `FlushD` y `FlushE`, que anulan las dos `addi ..., 99` que estaban en ID e IF. En el ciclo siguiente se ve el resultado: `InstrD` es un NOP, `InstrE` vale 0 y `PCF` ya apunta a `0x20` (`t1`). El `jal` de los 130 ns se comporta igual, y a los 150 ns `ResultW` muestra el 40 que se guarda en `x8` como dirección de retorno.

El caso más interesante es el `jalr` de los 180 ns. Recibe `x12 = 68` por forwarding desde MEM (`ForwardAE = 10`), pone `PCSrcE = 10` y esta vez sí provoca el flush, que es justamente lo que fallaba antes de la corrección. A los 190 ns el PC ya está en `0x44` (`t3`) y a los 200 ns `ResultW` muestra el 60 de `x13`.

El -2 que aparece en `ResultW` hacia los 90 ns no es un error: es la resta $5 - 7$ que hizo el `beq` no tomado al llegar a WB, y no se escribe en ningún registro porque `beq` tiene `RegWrite = 0`. Cada salto tomado cuesta dos ciclos, uno por cada instrucción anulada.

5. Primeros pasos hacia Dual-Issue

En esta entrega construimos la parte frontal del procesador Dual-Issue: leer y decodificar dos instrucciones por ciclo. La ejecución en paralelo (Issue Unit y segunda ALU) queda para E2. La Figura 6 resume los cambios.

Dual Fetch. A la memoria de instrucciones le agregamos un segundo puerto de lectura: `rd` entrega la instrucción en `PC` y `rd1` la de `PC+4`. Como en cada lectura llega un par, el PC avanza de 8 en 8. El PC se actualiza cuando el par que está en Decode ya terminó de emitirse o cuando hay un salto tomado.

```
assign rd  = RAM[a[31:2]];      // imem: instruccion en PC
assign rd1 = RAM[a[31:2] + 1]; // imem: instruccion en PC+4
.en(~StallF & (SelD | FlushD)) // habilitacion del registro del PC
```

Dual Decode. El registro IF/ID ahora es doble y guarda `InstrD0` (slot 0, la instrucción más antigua), `InstrD1` (slot 1) y el PC base del par. El controller tiene dos decodificadores (dos `maindec` y dos `aludec`) y el datapath dos unidades `extend`, así que ambas instrucciones se decodifican en el mismo ciclo. El Register File pasó a tener cuatro puertos de lectura y dos de escritura: lee `rs1` y `rs2` de las dos instrucciones a la vez. El segundo puerto de escritura ya existe y, si las dos vías escriben el mismo registro, gana la vía 1 por ser la instrucción más joven; lo conectaremos en E2.

```
always @(negedge clk) begin
  if (we3)  rf[a3] <= wd3;
  if (web3) rf[b3] <= wdb3;   // va despues: tiene prioridad
end
```

Módulos de la vía 1. También instanciamos el registro ID/EX de la vía 1 (`regE1`, con los operandos, el inmediato y el control del slot 1) y su ALU (`alu1`), con su propio multiplexor de `ALUSrc`. En esta entrega la señal `Issue1D` está fija en 0, de modo que ese registro se limpia en cada ciclo y la vía 1 solo procesa burbujas. En E2 la Issue Unit pondrá `Issue1D` en 1 cuando las dos instrucciones del par puedan salir juntas.

Emisión. Como E1 todavía no pide ejecutar dos instrucciones a la vez, emitimos una por ciclo y en orden. La señal `SelD` elige el slot: con `SelD = 0` sale la instrucción más antigua del par y con `SelD = 1` la más joven; después entra el siguiente par. A EX pasan el control y los operandos del slot elegido. Esta idea de guardar la instrucción más joven para el ciclo siguiente es la base del reagrupamiento que hará la Issue Unit en E2. Ante un salto tomado se limpia el par en Decode, y si el salto estaba en el slot 0, la instrucción del slot 1 se anula con `FlushE`.

```
assign InstrD = SelD ? InstrD1 : InstrD0;   // slot que se emite
assign PCD    = PCPairD + {SelD, 2'b00};
```

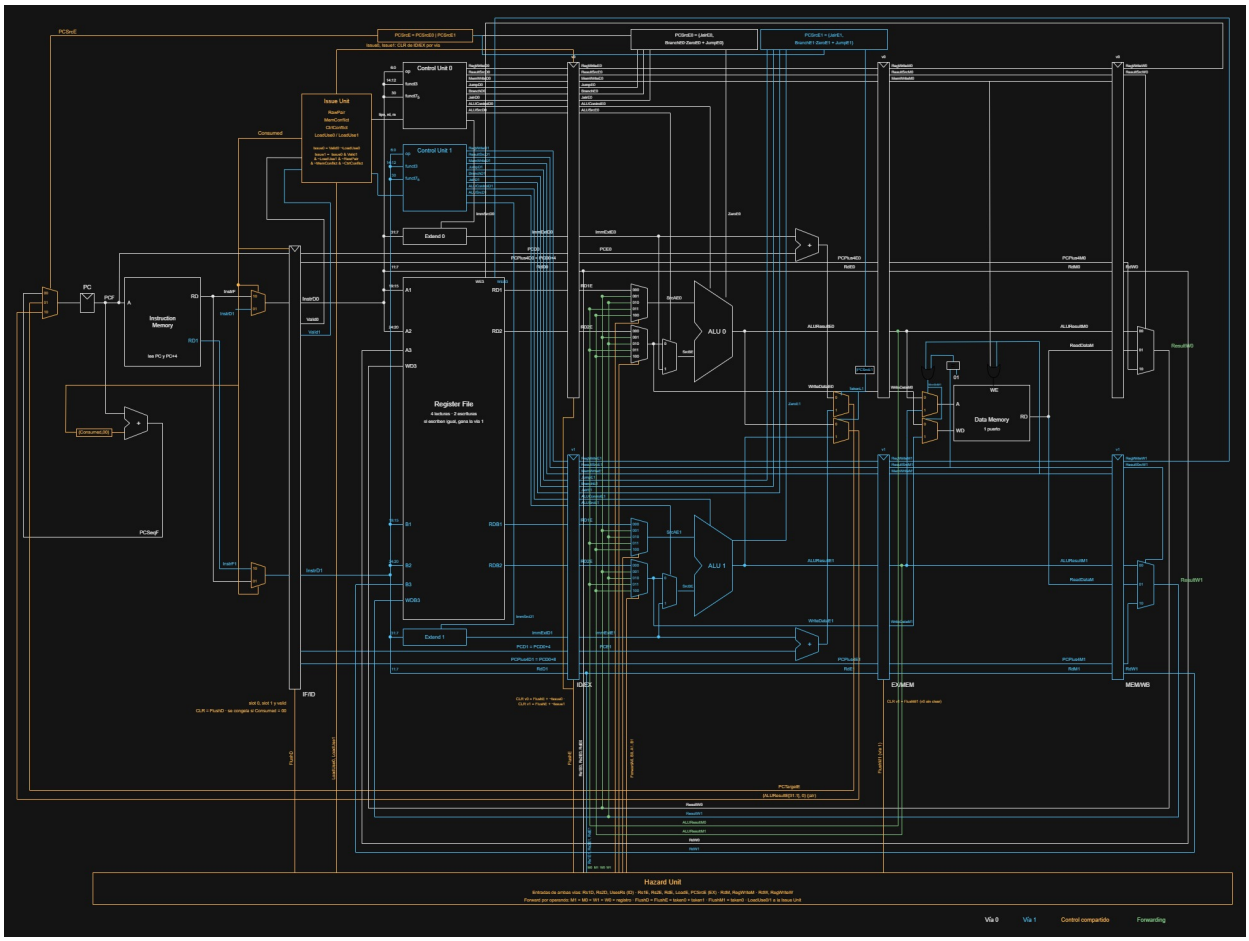


Figura 6: Datapath Dual-Issue preliminar. Arriba la vía 0 y abajo la vía 1; cada registro de pipeline está partido en una sección por vía. En blanco, lo que se mantiene del Single-Issue (Figura 1); en celeste, lo agregado en E1 para leer y decodificar el par; en gris punteado, la vía 1 después de Decode, que ya tiene su registro ID/EX y su ALU pero en E1 solo recibe burbujas.

5.1 Programa 5: evidencia de Dual Fetch y Dual Decode

```
        addi x1, x0, 1      # par 0x00
        addi x2, x0, 2
        addi x3, x0, 3      # par 0x08
        addi x4, x0, 4
        add  x5, x1, x2     # par 0x10
        add  x6, x3, x4
        sw   x5, 0(x0)      # par 0x18
        sw   x6, 4(x0)
        beq  x0, x0, t1     # par 0x20: salto tomado en el slot 0
        addi x7, x0, 99     #           el slot 1 se anula
t1:     addi x8, x0, 8
fin:    beq  x0, x0, fin
```

El testbench de este programa revisa cinco cosas: que el par leído sea siempre (PC, PC+4); que sin saltos el PC avance de 8 en 8; que después de cada salto el PC tome el destino correcto; que las 11 instrucciones válidas lleguen a EX en orden, sin perderse ni repetirse, y que registros y memoria terminen con los valores esperados. Los cuatro programas anteriores también pasan sin cambios sobre esta versión, así que el Dual Fetch y el Dual Decode no alteraron el comportamiento del procesador.

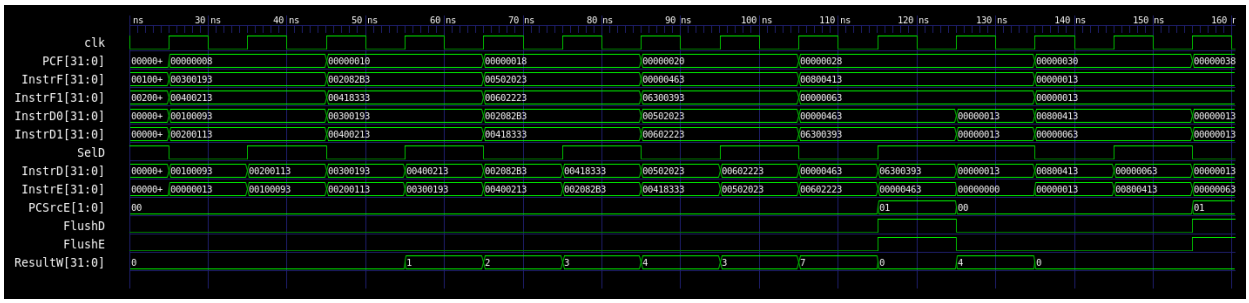


Figura 7: Programa 5, de 20 a 160 ns.

En la Figura 7 se ve el ritmo del Dual Fetch. A los 30 ns, Decode tiene el par `0x00` (`InstrD0 = addi x1`, `InstrD1 = addi x2`) y, al mismo tiempo, Fetch ya está leyendo el par `0x08` (`InstrF = addi x3`, `InstrF1 = addi x4`). Con `SelD = 0` sale `addi x1`; a los 40 ns, con `SelD = 1`, sale `addi x2`, mientras el PC y el par de Fetch esperan. A los 50 ns entra el par `0x08` a Decode y `PCF` salta de `0x08` a `0x10`. El patrón se repite cada dos ciclos: un par leído y decodificado, dos instrucciones emitidas.

El salto aparece a los 110 ns, cuando entra el par `0x20` con el `beq` en el slot 0 y `addi x7, 99` en el slot 1. El `beq` se emite primero y a los 120 ns se resuelve tomado en EX (`PCSrcE = 01`) justo cuando se está emitiendo `addi x7`. `FlushE` la anula (a los 130 ns `InstrE` vale 0) y `FlushD` limpia el par. A los 140 ns entra el par destino (`addi x8` y `beq fin`) y todo sigue en orden.

Un error que encontramos con este programa

Al revisar la simulación vimos que, si un salto se tomaba mientras se emitía el slot 0, el PC no saltaba: solo se actualizaba cuando `SelD = 1`, y el destino se perdía. Lo corregimos permitiendo que el PC se actualice también cuando hay un salto tomado (`en = ~StallF & (SelD | FlushD)`), y agregamos al testbench la verificación del destino después de cada salto, que con la versión anterior falla.

6. Conclusiones

El pipeline de cinco etapas ejecuta correctamente el subconjunto RV32I que soporta y resuelve los tres riesgos del pipeline: forwarding desde MEM y WB, un ciclo de stall ante load-use y la anulación de dos instrucciones en cada salto tomado. Las pruebas automáticas fueron clave: gracias a ellas encontramos dos

errores, el flush que faltaba en `jalr` y la pérdida del destino del salto en el Dual Fetch, que con una revisión visual de las waveforms habrían sido fáciles de pasar por alto.

La base del Dual-Issue ya está: Fetch y Decode trabajan con pares de instrucciones y el Register File tiene los cuatro puertos de lectura y dos de escritura que se van a necesitar. La vía 1 ya tiene su registro ID/EX y su ALU. Para E2 queda lo más interesante: completar esa vía hasta WB, la Issue Unit que decida si dos instrucciones pueden salir juntas (dependencias RAW dentro del par y recursos compartidos de memoria y branch) y el forwarding entre vías.

A. Cómo reproducir las pruebas en Vivado

El código entregado tiene dos carpetas: `Single_Issue`, con la arquitectura A, y `Dual_Issue`, con el avance hacia B. Dentro de cada una hay una carpeta por programa con su código Assembly, su archivo `.mem` y su testbench (Tabla 4). Los programas 1 a 4 también pasan sin cambios en `Dual_Issue`.

Tabla 4: Archivos de cada prueba.

Programa	Código	Carpeta	Testbench	Programa cargado
1 (ISA)	`Single_Issue`	`Programa1_ISA`	`tb_isa.v`	`isa.mem`
2 (Forwarding)	`Single_Issue`	`Programa2_Forwarding`	`tb_forwarding.v`	`forwarding.mem`
3 (Stalling)	`Single_Issue`	`Programa3_Stalling`	`tb_stalling.v`	`stalling.mem`
4 (Flushing)	`Single_Issue`	`Programa4_Flushing`	`tb_flushing.v`	`flushing.mem`
5 (Dual Fetch)	`Dual_Issue`	`Programa5_DualFetch`	`tb_dualfetch.v`	`dualfetch.mem`

Los programas 1 a 4 comparten un mismo proyecto de Vivado. El Programa 5 necesita otro, porque usa el código de `Dual_Issue`.

Crear el proyecto.

1. Crear un proyecto RTL nuevo en Vivado.
2. En *Add Sources* → *design sources*, agregar todos los `.v` sueltos de `Single_Issue` (los que no están dentro de una carpeta `Programa`). Para el Programa 5 se usan los de `Dual_Issue`.

Simular un programa.

1. En *Add Sources* → *simulation sources*, agregar el testbench y el `.mem` del programa según la Tabla 4. Para el Programa 1 son `Programa1_ISA/tb_isa.v` y `Programa1_ISA/isa.mem`.
2. Comprobar que el testbench quedó como top: en *Simulation Sources* aparece en negrita. Si hay más de un testbench en el proyecto, Vivado puede haber elegido otro; en ese caso, clic derecho sobre el correcto y *Set as Top*.
3. *Run Simulation* → *Run Behavioral Simulation*. La simulación termina sola y la consola Tcl muestra `PASS` o `FAIL`.
4. Cerrar la simulación (*Close Simulation*) y repetir los pasos con el siguiente programa de la tabla.

La waveform ya muestra `PCF`, `InstrF` a `InstrW` y las señales de la Hazard Unit, porque el testbench las declara en su nivel superior, así que no hay que agregarlas a mano.

Entrega 2

1. Arquitectura Dual-Issue

En esta entrega el procesador ya ejecuta dos instrucciones por ciclo cuando es posible. Conserva las cinco etapas y emite en orden: la instrucción más antigua (slot 0) va por la vía 0 y la más joven (slot 1) por la vía 1. Los sufijos 0 y 1 de las señales indican la vía. La Figura 8 muestra el datapath completo.

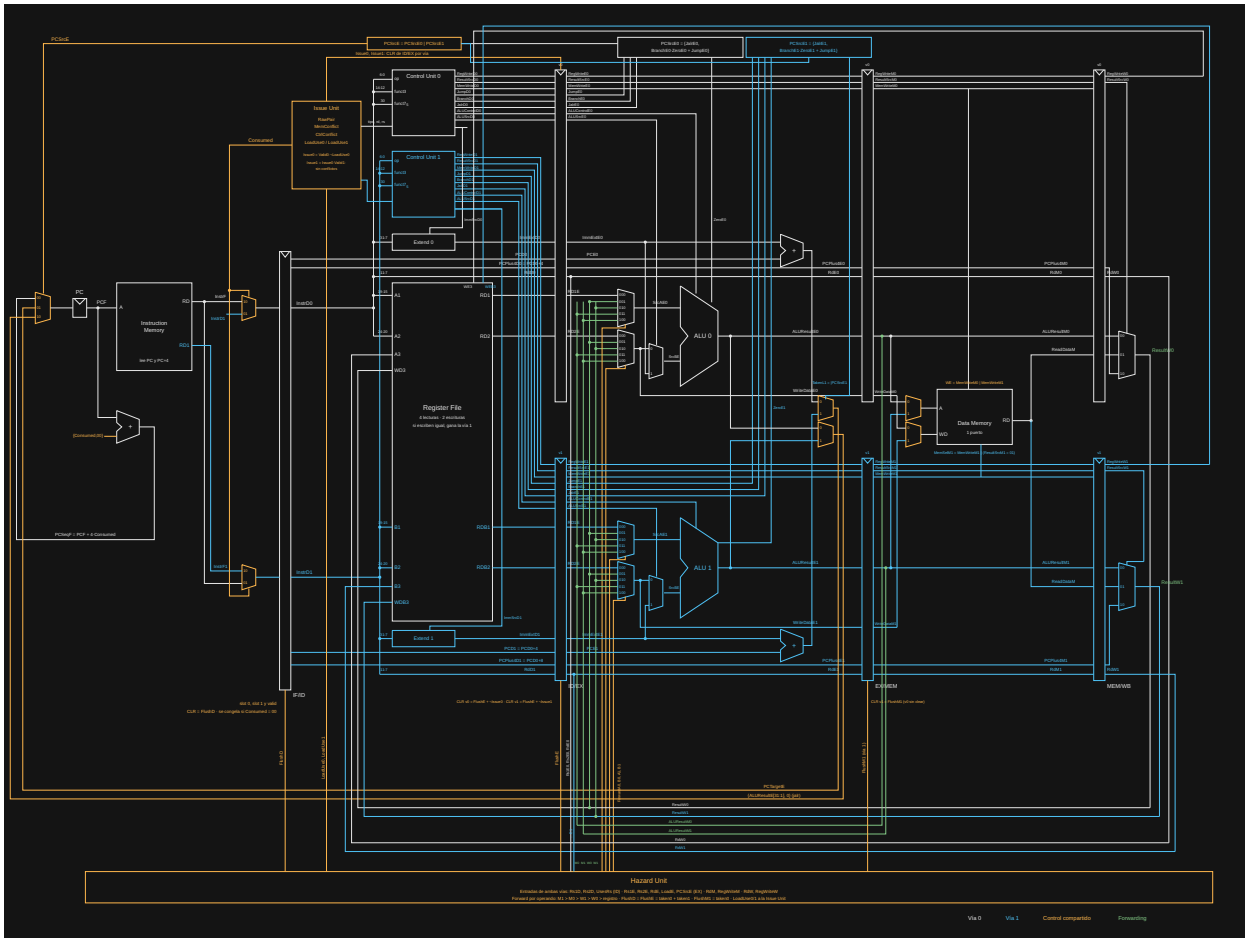


Figura 8: Datapath Dual-Issue completo. Arriba la vía 0 (blanco) y abajo la vía 1 (celeste), cada una con sus propios registros ID/EX, EX/MEM y MEM/WB; el registro IF/ID guarda el par de instrucciones. En naranja, la Issue Unit, la Hazard Unit y lo que comparten las dos vías: el resolutor de saltos y la memoria de datos. En verde, los caminos de forwarding.

Fetch. Se leen siempre dos instrucciones consecutivas, y el PC avanza 4 bytes por cada instrucción consumida:

```
adder pcadd(.a(PCF), .b({28'b0, Consumed, 2'b00}), .y(PCSeqF));
```

IF/ID como ventana deslizante. El registro guarda dos instrucciones y un bit de validez por slot. Reemplaza al `SelD` de E1: si solo sale el slot 0, el slot 1 pasa a ser el slot 0 y se reagrupa con la instrucción siguiente:

```
end else if (Consumed == 2'b10) begin       // salieron los dos slots
  InstrD0_r <= InstrF;  InstrD1_r <= InstrF1;  PCD0_r <= PCF;
end else if (Consumed == 2'b01) begin       // solo salio el slot 0
  InstrD0_r <= InstrD1_r;  InstrD1_r <= InstrF;  PCD0_r <= PCD0_r + 4;
end                                         // Consumed = 0: la ventana se congela
```

Recursos por vía. Hay dos ALU, dos registros ID/EX, EX/MEM y MEM/WB, y un Register File de cuatro lecturas y dos escrituras (Sección 5 de la Entrega 1). Si ambas vías escriben el mismo registro, gana la vía 1.

Recursos compartidos. La memoria de datos tiene un solo puerto, y un multiplexor se lo entrega a la vía que tenga el load o store. También hay un solo resolutor de saltos, que toma el destino de la vía que tiene el salto:

```
assign MemSelM1 = MemWriteM1 | (ResultSrcM1 == 2'b01);  // puerto de memoria
wire   TakenL1  = |PCSrcE1;                             // destino del salto
```

Controller. Decodifica las dos instrucciones y lleva un registro de control por vía. Además informa a la Issue Unit qué registros usa cada instrucción, para no crear dependencias falsas: `addi`, por ejemplo, no usa `rs2`. La principal novedad respecto a E1 es que cada vía calcula su propio `PCSrc`.

Tabla 5: Módulos nuevos y modificados respecto a E1.

Módulo	Estado	Cambio
`issueunit.v`	Nuevo	Decide la emisión y cuántos slots avanza la ventana.
`fwdsel.v`	Nuevo	Selector de forwarding de un operando.
`mux5.v`	Nuevo	Multiplexor de forwarding de cinco entradas.
`datapath.v`	Modificado	Ventana IF/ID, dos vías completas y puerto de memoria compartido.
`controller.v`	Modificado	Control por vía y datos para la Issue Unit.
`hazardunit.v`	Modificado	Cuatro forwardings, load-use por slot y flush por vía.
`riscvpipeline.v`	Modificado	Conecta la Issue Unit y las señales de las dos vías.
`imem.v`, `dmem.v`	Modificado	Memorias ampliadas a 256 palabras.

2. Issue Unit

La Issue Unit (`issueunit.v`) decide en ID cuántas instrucciones del par salen en el ciclo. El slot 1 nunca sale sin el slot 0, así se conserva el orden del programa:

```
assign RawPair = RegWriteD0 & (RdD0 != 5'b0) &
                 ((UsesRs1D1 & (Rs1D1 == RdD0)) | (UsesRs2D1 & (Rs2D1 == RdD0)));
assign MemConflict  = IsMemD0  & IsMemD1;
assign CtrlConflict = IsCtrlD0 & IsCtrlD1;

assign Issue0 = Valid0 & ~LoadUse0;
assign Issue1 = Issue0 & Valid1 & ~LoadUse1 & ~RawPair & ~MemConflict & ~CtrlConflict;
```

Tabla 6: Condiciones que impiden emitir el par.

Señal	Condición	Efecto
`RawPair`	El slot 1 lee el registro que escribe el slot 0.	Sale solo el slot 0
`MemConflict`	Los dos son load o store: hay un solo puerto.	Sale solo el slot 0
`CtrlConflict`	Los dos son saltos: hay un solo resolutor.	Sale solo el slot 0
`LoadUse1`	El slot 1 usa el dato de un `lw` que está en EX.	Sale solo el slot 0
`LoadUse0`	El slot 0 usa el dato de un `lw` que está en EX.	No sale ninguno

Asignación de recursos. La instrucción se queda en la vía de su slot, sin importar su tipo. Como ambas vías tienen ALU y acceso al puerto de memoria y al resolutor, una ALU puede salir junto con un load o store, o con un salto, en cualquier orden.

Reagrupamiento. La señal `Consumed` indica cuántas posiciones avanza la ventana. Un slot inválido cuenta como consumido, y por eso la ventana se recarga sola después de un reset o un flush:

```
wire consumed0 = Issue0 | ~Valid0;
wire consumed1 = Issue1 | ~Valid1;
assign Consumed = consumed0 ? (consumed1 ? 2'b10 : 2'b01) : 2'b00;
```

3. Forwarding, stalls y control de saltos

Forwarding. Cada uno de los cuatro operandos de EX tiene un multiplexor de cinco entradas y su propio selector. Cuando varios productores escriben el mismo registro, gana el más reciente: primero MEM y luego WB, y dentro de cada etapa, la vía 1:

```
if      ((Rs != 5'b0) && RegWriteM1 && (Rs == RdM1)) Forward = 3'b100; // MEM via 1
else if ((Rs != 5'b0) && RegWriteM0 && (Rs == RdM0)) Forward = 3'b011; // MEM via 0
else if ((Rs != 5'b0) && RegWriteW1 && (Rs == RdW1)) Forward = 3'b010; // WB via 1
else if ((Rs != 5'b0) && RegWriteW0 && (Rs == RdW0)) Forward = 3'b001; // WB via 0
else                                                 Forward = 3'b000; // registro
```

Stall por load-use. El dato de un `lw` recién está listo al final de MEM. Se compara cada slot en ID con los dos posibles `lw` en EX; este es el caso del slot 0:

```
assign LoadUse0 = (ldE0 & ((UsesRs1D0 & (Rs1D0 == RdE0)) | (UsesRs2D0 & (Rs2D0 == RdE0))))
                | (ldE1 & ((UsesRs1D0 & (Rs1D0 == RdE1)) | (UsesRs2D0 & (Rs2D0 == RdE1))));
```

Si `LoadUse0` vale 1, `Consumed` es 0: el PC y la ventana se mantienen, y al no emitirse nada, EX recibe burbujas. Un salto tomado tiene prioridad sobre el stall.

Saltos y flush. El salto se resuelve en EX, en cualquiera de las dos vías. Solo se anula lo más joven que el salto:

```
assign FlushD  = taken0 | taken1;  // IF/ID
assign FlushE  = taken0 | taken1;  // ID/EX de las dos vias
assign FlushM1 = taken0;           // EX/MEM de la via 1
```

Si el salto está en la vía 0, la instrucción de la vía 1 es más joven y `FlushM1` impide que llegue a MEM. Si está en la vía 1, la vía 0 es más antigua y continúa normalmente.

4. Programas de prueba y waveforms

Los programas de esta entrega están en `Dual_Issue`. Cada testbench comprueba el orden de retiro con el PC de cada instrucción que llega a WB, los registros y la memoria finales, y un número mínimo de pares retirados. La Tabla 7 los relaciona con lo que debe probarse. Las 13 pruebas dan PASS. Para facilitar la lectura de las waveforms, las señales en amarillo son datos en decimal, las instrucciones aparecen en ensamblador (`(burbuja)` es una etapa vacía) y cada señal `Forward` indica también el origen del dato: `reg`, `WB0`, `WB1`, `MEM0` o `MEM1`.

Tabla 7: Programas de prueba de la Entrega 2.

Programa	Qué prueba	Figura
6 — Dos ALU	Dos operaciones ALU independientes emitidas juntas	9
7 — RAW y forwarding	RAW entre candidatas y forwarding entre vías y etapas	10
8 — Load y memoria	Load-use y conflictos del puerto de memoria	11
9 — Branches	Saltos tomados y no tomados, `jal` y `jalr`	—
10 — Escrituras	Dos escrituras en el Register File y al mismo registro	14
11 — Reagrupamiento	Emisión parcial, orden y ausencia de pérdidas o duplicados	15
12 — Benchmark	Programa con lazos para comparar rendimiento	—
13 — Casos vía 1	Load-use en el slot 1; salto no tomado y `jalr` en la vía 1	12, 13

4.1 Dos operaciones ALU juntas

```
# Programa 6: dos ALU independientes salen juntas (Issue1 = 1)
        addi x1, x0, 1
        addi x2, x0, 2
        addi x3, x0, 3
        addi x4, x0, 4
        addi x5, x0, 5
        addi x6, x0, 6
        addi x7, x0, 7
        addi x8, x0, 8
        add  x9, x1, x2
        sub  x10, x3, x4
        and  x11, x5, x6
        or   x12, x7, x8
        xor  x13, x1, x3
        slt  x14, x2, x4
        addi x15, x5, 10
        addi x16, x6, 20
fin:    beq  x0, x0, fin
```



Figura 9: Programa 6, de 55 a 115 ns.

En la Figura 9, `Issue1` permanece en 1 y `Consumed = 10`: sale un par en cada ciclo. A los 70 ns, Decode tiene `add x9, x1, x2` y `sub x10, x3, x4`. A los 80 ns ambas están en EX: la ALU 0 calcula $1 + 2 = 3$ y la ALU 1 calcula $3 - 4 = -1$. A los 100 ns `ResultW0` y `ResultW1` entregan esos dos resultados en el mismo ciclo.

4.2 RAW dentro del par y forwarding entre vías

```
# RAW dentro del par (no se emparejan) y forwarding entre vias y etapas
        addi x1, x0, 5
        addi x2, x0, 7
        add  x3, x1, x2        # usa x1 y x2 recien calculados: forwarding
        add  x4, x3, x1        # RAW con la instruccion anterior (x3)
        sub  x5, x4, x3
        and  x6, x5, x4
        or   x7, x6, x5
        xor  x8, x7, x6
        slt  x9, x8, x7
        addi x10, x0, 10       # dos escrituras al mismo registro: gana la mas joven
        addi x10, x0, 20
        add  x11, x10, x10     # debe leer 20
        add  x12, x11, x9
        add  x13, x12, x11
        sw   x13, 0(x0)
fin:    beq  x0, x0, fin
```

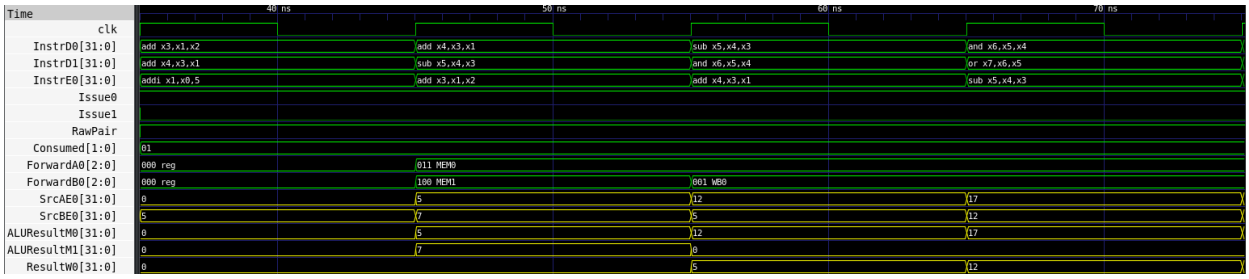


Figura 10: Programa 7, de 35 a 75 ns.

En la Figura 10, a los 40 ns el par es `add x3, x1, x2` y `add x4, x3, x1`. La segunda necesita `x3`, así que `RawPair = 1`, `Issue1 = 0` y `Consumed = 01`. A los 50 ns, la `add x4` ya está en el slot 0 y `add x3` en EX recibe sus operandos de las dos vías de MEM: `ForwardA0 = 011` da el 5 de `ALUResultM0` y `ForwardB0 = 100` da el 7 de `ALUResultM1`. A los 60 ns, `add x4` toma el 12 desde MEM y el 5 desde WB (`ForwardB0 = 001`).

4.3 Conflicto de memoria y load-use

```
# load-use y conflictos del puerto de memoria
        addi x1, x0, 11
        addi x2, x0, 22
        sw   x1, 0(x0)
        sw   x2, 4(x0)         # sw + sw: un solo puerto, no se emparejan
        lw   x3, 0(x0)
        add  x4, x1, x2        # lw + ALU independiente: pueden salir juntas
        add  x5, x1, x1
        lw   x6, 4(x0)         # ALU + lw: tambien en el otro orden
        add  x7, x6, x3        # load-use: depende del lw anterior
        sub  x8, x7, x3
        sw   x8, 8(x0)
        lw   x9, 8(x0)
        addi x10, x9, 1        # load-use inmediato
        add  x11, x10, x4
        sw   x11, 12(x0)
fin:    beq  x0, x0, fin
```

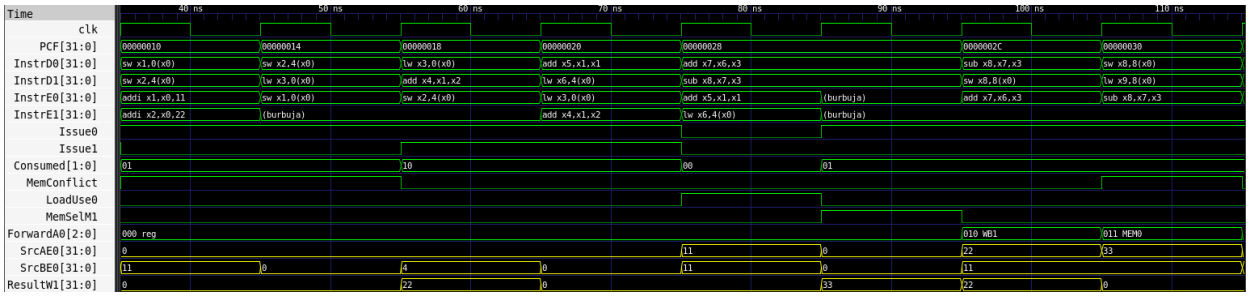


Figura 11: Programa 8, de 35 a 115 ns.

En la Figura 11, a los 40 ns el par son dos `sw`: `MemConflict = 1` y salen en ciclos distintos. A los 60 ns salen juntas `lw x3` y `add x4`, y a los 70 ns, en el orden inverso, `add x5` y `lw x6`, lo que muestra la memoria junto con la ALU en ambas posiciones. Por eso a los 90 ns `MemSelM1 = 1`: el puerto es de la vía 1.

A los 80 ns `add x7, x6, x3` necesita el `lw x6` que está en EX: `LoadUse0 = 1`, `Issue0 = 0` y `Consumed = 00`. A los 90 ns el PC sigue en `0x28`, Decode no cambió y EX tiene burbujas. A los 100 ns, `ForwardA0 = 010` entrega el 22 desde `ResultW1`, que es el dato leído por la vía 1.

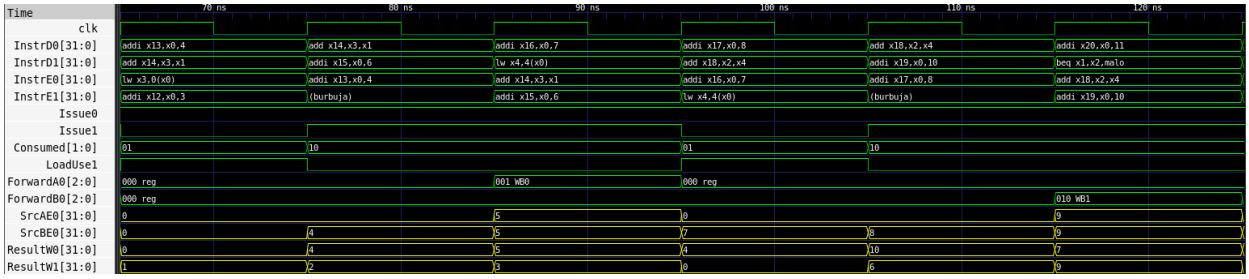


Figura 12: Programa 13, load-use en el slot 1, de 65 a 125 ns.

La Figura 12 muestra el load-use en el slot 1. A los 70 ns, `add x14, x3, x1` depende de `lw x3`: `LoadUse1 = 1`, pero el slot 0 es independiente y sale solo (`Consumed = 01`). A los 90 ns, `ForwardA0 = 001` entrega el 5 leído desde `ResultW0`. A los 100 ns ocurre lo mismo con un `lw` en la vía 1, y a los 120 ns el 9 llega por `ResultW1` (`ForwardB0 = 010`).

4.4 Saltos en ambas posiciones y flushing

```
# branches en cualquiera de las dos posiciones del par, tomados y no tomados, jal y jalr
        addi x1, x0, 1
        addi x2, x0, 1
        addi x3, x0, 2
        beq  x1, x3, ta         # no tomado
        addi x4, x0, 4
        beq  x1, x2, tb         # tomado: se anula solo lo mas joven
        addi x5, x0, 99
        addi x6, x0, 99
tb:     addi x7, x0, 7
        addi x8, x0, 8
        beq  x7, x8, tc         # no tomado
        addi x9, x0, 9
        nop
        beq  x1, x2, td         # tomado (otra posicion dentro del par)
        addi x11, x0, 99
        addi x12, x0, 99
ta:     addi x13, x0, 99
tc:     addi x14, x0, 99
td:     addi x15, x0, 15
        jal  x16, te            # salto incondicional
        addi x17, x0, 99
        addi x18, x0, 99
te:     addi x19, x0, 104
        jalr x20, 0(x19)        # salto indirecto
        addi x21, x0, 99
        addi x22, x0, 99
tf:     addi x23, x0, 23
        sw   x23, 0(x0)
fin:    beq  x0, x0, fin
```

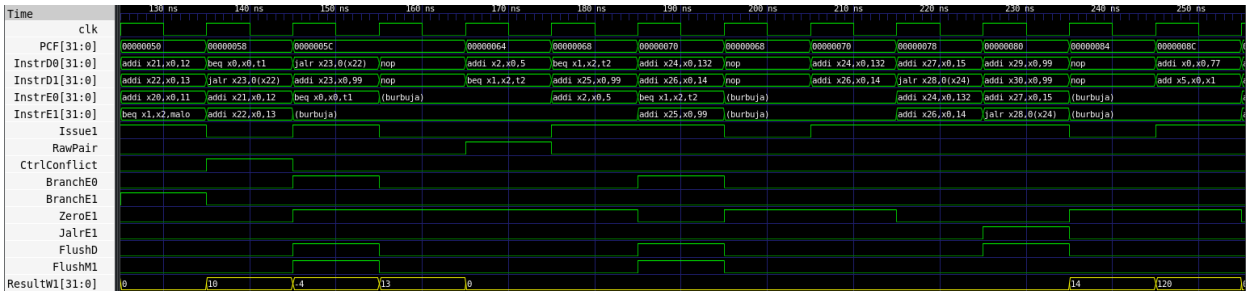


Figura 13: Programa 13, saltos, de 125 a 255 ns.

En la Figura 13 aparecen los tres casos:

- **No tomado en la vía 1 (130 ns).** `BranchE1 = 1` y `ZeroE1 = 0`, porque $5 \neq 9$. No hay flush y el programa sigue.
- **Dos saltos en el par (140 ns).** `beq` y `jalr` levantan `CtrlConflict`. Sale solo el `beq`, que se toma a los 150 ns: `FlushD` y `FlushM1` anulan todo lo posterior. A los 190 ns ocurre otro salto tomado en la vía 0, y `FlushM1` anula la `addi x25` que iba en la vía 1.
- **`jalr` en la vía 1 (230 ns).** `JalrE1 = 1` y `FlushD = 1`, pero `FlushM1 = 0`: la `addi x27` de la vía 0 es más antigua y continúa. El PC pasa a `0x84` y a los 250 ns `ResultW1` escribe el enlace 120 (`0x74 + 4`).

4.5 Dos escrituras en el Register File

```
# dos escrituras al Register File por ciclo y escrituras al mismo registro
        addi x1, x0, 1          # x1 y x2 se escriben en el mismo ciclo (puertos 0 y 2)
        addi x2, x0, 2
        addi x3, x0, 3
        addi x4, x0, 4
        addi x5, x0, 5          # WAW: dos escrituras al mismo registro; gana la mas joven
        addi x5, x0, 6
        addi x6, x0, 7
        addi x6, x0, 8
        add  x7, x5, x6         # 6 + 8
        add  x8, x1, x2
        sw   x7, 0(x0)
        sw   x8, 4(x0)
fin:    beq  x0, x0, fin
```

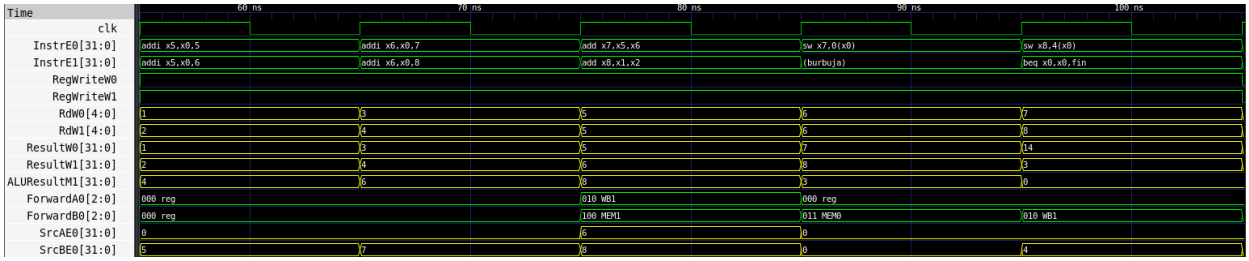


Figura 14: Programa 10, de 55 a 105 ns.

En la Figura 14, `RegWriteW0` y `RegWriteW1` valen 1 a la vez: a los 60 ns se escriben `x1 = 1` y `x2 = 2` en el mismo ciclo. A los 80 ns las dos vías escriben `x5` (5 y 6) y debe ganar la más joven. Justo entonces, `add x7, x5, x6` está en EX y `ForwardA0 = 010` le entrega el 6 de la vía 1. Además, `ForwardB0 = 100` le entrega el 8 de la vía 1 de MEM, y no el 7 de la vía 0, por la misma prioridad.

4.6 Reagrupamiento y orden de retiro

```
# emision parcial y reagrupamiento (orden preservado, sin instrucciones perdidas ni repetidas)
        addi x1, x0, 3
        add  x2, x1, x1         # RAW: sale sola, la siguiente se reagrupa
        add  x3, x2, x1
        addi x4, x0, 4          # independiente: puede emparejarse con la anterior
        add  x5, x4, x3
        sw   x5, 0(x0)
        lw   x6, 0(x0)
        sw   x6, 4(x0)          # RAW + memoria
        addi x7, x0, 7
        addi x8, x0, 8
        add  x9, x7, x8
        sub  x10, x9, x7
        sw   x10, 8(x0)
fin:    beq  x0, x0, fin
```

En la Figura 15, de 30 a 40 ns hay `RawPair` y `Consumed = 01`. `InstrD1` pasa a `InstrD0` en el ciclo siguiente y el PC avanza solo 4 (`0x08` a `0x0C`). A los 50 ns se forma un par nuevo con `add x3` y `addi x4`, instrucciones que no venían juntas de Fetch, y salen las dos (`Consumed = 10`, el PC avanza 8). A los 90 ns un load-use congela la ventana (`Consumed = 00`). `PCW0` y `PCW1` muestran el retiro en orden: `0x00`, `0x04`, el par `0x08` y `0x0C`, y luego `0x10` y `0x14`, sin pérdidas ni repeticiones.

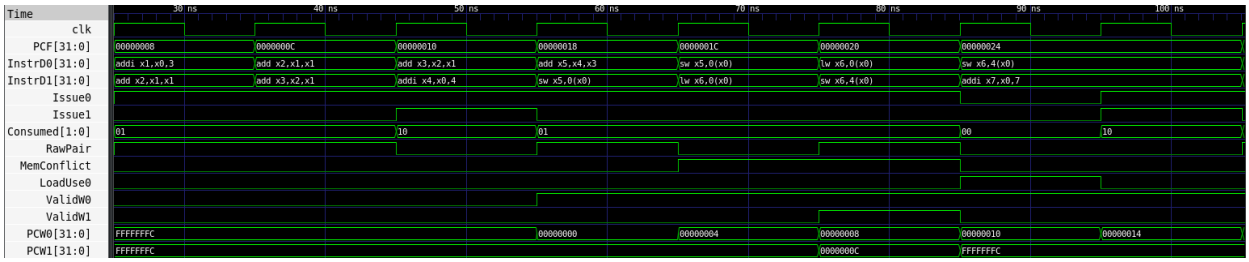


Figura 15: Programa 11, de 25 a 105 ns. `PCW = FFFFFFFC` indica que no se retira ninguna instrucción.

5. Comparación con el Single-Issue

La Tabla 8 compara las arquitecturas A (Single-Issue) y B (Dual-Issue) con los mismos programas. Los ciclos se cuentan igual en ambas, desde que baja el reset hasta que se retira la última instrucción del programa, y en los dos casos se comprueba el mismo resultado final. Los testbenches de la arquitectura A están en `Single_Issue` con el sufijo `_single`; ambas versiones se simulan con los pasos del Anexo A.

Tabla 8: Ciclos e IPC preliminares. Speedup = ciclos A / ciclos B.

Programa	Instr.	Ciclos A	Ciclos B	IPC A	IPC B	Speedup
1 — ISA	29	32	19	0.91	1.53	1.68
2 — Forwarding	7	10	8	0.70	0.88	1.25
3 — Stalling	8	12	10	0.67	0.80	1.20
4 — Flushing	14	23	18	0.61	0.78	1.28
5 — Dual Fetch	11	16	11	0.69	1.00	1.45
6 — Dos ALU	17	20	12	0.85	1.42	1.67
7 — RAW	16	19	16	0.84	1.00	1.19
8 — Load y memoria	16	21	17	0.76	0.94	1.24
9 — Branches	19	30	23	0.63	0.83	1.30
10 — Escrituras	13	16	10	0.81	1.30	1.60
11 — Reagrupamiento	14	18	15	0.78	0.93	1.20
12 — Benchmark	110	153	133	0.72	0.83	1.15
13 — Casos vía 1	34	46	31	0.74	1.10	1.48

El mayor speedup aparece donde hay instrucciones independientes: 1.68 en el Programa 1 y 1.67 en el 6. El menor es el del benchmark, 1.15. En sus 110 instrucciones solo se retiraron 20 pares y 70 instrucciones sueltas, con 8 stalls y 17 flushes. Sus lazos tienen dependencias RAW encadenadas y terminan en `beq` y `jal` seguidos, que provocan flushes y no pueden emitirse juntos. Estas son las causas que impiden llegar al IPC ideal de 2.

5.1 Cómo reproducir las pruebas

Para simular cada programa se siguen los pasos del Anexo A, con el código de `Dual_Issue` o `Single_Issue` y los archivos de la Tabla 9. En `Single_Issue`, el testbench lleva el sufijo `_single`.

Tabla 9: Archivos de cada programa de la Entrega 2.

Programa	Carpeta	Testbench	Programa cargado
1	`Programa1_ISA`	`tb_isa.v`	`isa.mem`
2	`Programa2_Forwarding`	`tb_forwarding.v`	`forwarding.mem`
3	`Programa3_Stalling`	`tb_stalling.v`	`stalling.mem`
4	`Programa4_Flushing`	`tb_flushing.v`	`flushing.mem`
5	`Programa5_DualFetch`	`tb_dualfetch.v`	`dualfetch.mem`
6	`Programa6_DosALU`	`tb_dos_alu.v`	`dos_alu.mem`
7	`Programa7_RAW_Forwarding`	`tb_raw_forwarding.v`	`raw_forwarding.mem`
8	`Programa8_Load_Memoria`	`tb_load_memoria.v`	`load_memoria.mem`
9	`Programa9_Branches`	`tb_branches.v`	`branches.mem`
10	`Programa10_Escrituras`	`tb_escrituras.v`	`escrituras.mem`
11	`Programa11_Reagrupamiento`	`tb_reagrupamiento.v`	`reagrupamiento.mem`
12	`Programa12_Benchmark`	`tb_benchmark.v`	`benchmark.mem`
13	`Programa13_Casos_Via1`	`tb_casos_via1.v`	`casos_via1.mem`